Le Data Mining

Cours 5 — Du texte aux vecteurs

Christopher de Boisvilliers

2026-11-02

Plan de la séance

1. **Pourquoi le langage résiste**
2. **Du texte aux tokens**
3. **Distance d'édition**
4. **Sac de mots et cosinus**
5. **TF-IDF**
6. **Des mots aux vecteurs (Word2Vec)**
7. **Limites des méthodes vues**

Les 5 séances

5 · NLP : du texte aux vecteurs

6 · Embeddings et Transformers

7 · Recherche vectorielle

8 · RAG avec LangChain

9 · Évaluer et industrialiser

Aujourd'hui : **2 h 30 de cours** — puis **1 h de TP**.
Objectif : savoir transformer un texte en nombres, et savoir *pourquoi* chaque transformation change le résultat.

Objectifs de la séance

- **Préparer** un texte : normaliser, découper, filtrer
- **Comparer** deux textes : distance d'édition, cosinus
- **Construire** des vecteurs : comptages, TF-IDF
- **Comprendre** le principe de Word2Vec
- **Choisir** une représentation selon la question posée

Aucune connaissance en linguistique n'est nécessaire : tout part d'exemples chiffrés.

Deux phrases, une même scène

> « Le chat dort sur le canapé. »

> « Un félin se repose sur le sofa. »

13 occurrences · 10 formes · **2 formes communes** (`le` , `sur`) · **0 mot de contenu commun**
« mot de contenu » = un mot qui porte du sens (nom, verbe, adjectif), par opposition aux mots outils.

Pour un humain, c'est la même scène.
Pour une méthode qui compte les mots, la similarité est faible et ne repose que sur **deux mots outils**.

C'est ce gouffre que ce cours comble, une brique à la fois.

Pourquoi est-ce si difficile ?

- **Plusieurs sens** : `avocat` = un fruit ou un métier
- **Mots nouveaux** : `googliser` , `chanmé`
- **Écritures différentes** : `été` , `ÉTÉ` , `ete`
- **Synonymes** : `voiture` et `automobile` ne se ressemblent pas
- **Contexte** : le sens de `banque` dépend de la phrase

Un même mot ne portant pas toujours le même sens, aucun comptage ne suffit.

Quatre niveaux d'analyse

À ne pas confondre avec les quatre difficultés : ici on regarde *où* se situe le problème, sur **quatre niveaux retenus dans ce cours**.

Morphologique
La forme des mots
`chats = chat + s (pluriel)`

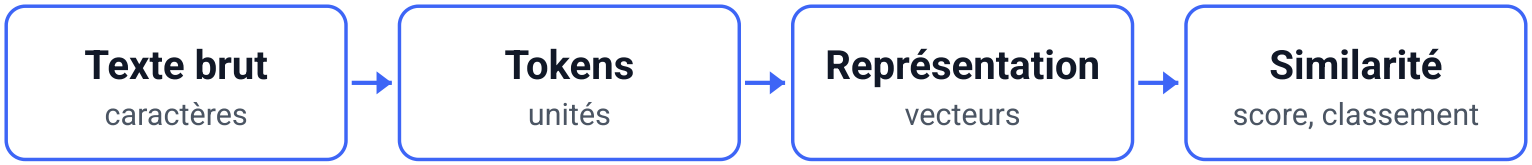
Syntaxique
La structure de la phrase
`[Les chats] sujet — [dorment] verbe`

Sémantique
Le sens littéral
`dormir( chats ) → état`

Discursif
L'enchaînement
`« Les chats dorment. Ils sont fatigués. »`

Le NLP classique les traite séparément. La suite du parcours montrera des modèles qui les traitent ensemble.

La chaîne que nous allons construire



Le contexte, lui, sera encodé dans la représentation à partir de la séance 6.

Aujourd'hui
Texte → tokens → vecteurs

Séances 6 à 9
Contexte → index → génération

Chaque brique sera construite à la main avant d'être appelée via une bibliothèque.

De la chaîne de caractères aux tokens

Le vocabulaire de base

Terme	Définition
Type	forme distincte
Occurrence	apparition d'un type dans le texte
Token	unité produite par le tokenizer (ici : un mot)
Vocabulaire V	ensemble des types
Identifiant	entier associé à un type

`le chat et le chien`

- **5 tokens** (occurrences)
- **4 types** (`le` apparaît 2 fois)
- V = { `le` , `chat` , `et` , `chien` }
- identifiants : `le` →0, `chat` →1, `et` →2, `chien` →3

Un modèle ne voit jamais de texte : il voit des **identifiants**. Toute la question est de savoir ce qu'on met *derrière* ces identifiants.
Dans tout ce cours, **token = un mot**. En séance 6, le token deviendra un **morceau de mot** : c'est le même terme, mais pas la même unité.

Un caractère n'est pas toujours un caractère

```
>>> len("é")                            # forme composée (NFC)
1
>>> len(unicodedata.normalize("NFD", "é")) # e + accent combinant
2
```

- `len()` compte des **points de code**, pas des caractères perçus (une lettre accentuée peut en occuper deux).
- Conséquence : deux textes visuellement identiques ne sont pas forcément égaux caractère par caractère.
- D'où, souvent, une **normalisation avant de comparer** — mais un tokenizer préentraîné peut imposer la sienne.

Règle pratique : `unicodedata.normalize("NFC", texte)` avant de comparer deux chaînes — sinon, suivre la normalisation du tokenizer utilisé.

Normaliser, c'est décider ce qu'on perd

Décision	Quand la prendre	Ce qu'on perd
minuscules	presque toujours	la casse (`Apple ≠ apple`)
supprimer les accents	recherche tolérante	`cote` / `côté` / `coté`
retirer la ponctuation	sac de mots	le point d'interrogation, l'emphase
retirer les chiffres	textes littéraires	`30 %` , `COVID-19`
espaces multiples	texte continu	l'indentation d'un code

Il n'existe pas de « bonne » normalisation : il existe des **choix**, et chacun se paie quelque part.

Un tokenizer préentraîné (BERT, GPT…) applique déjà sa propre normalisation : la refaire peut la contredire.
Règle simple : comparer deux chaînes → les normaliser *toutes les deux* dans la même forme (NFC par défaut, NFKC si ligatures ou compatibilité) ; passer du texte à un tokenizer préentraîné → ne rien normaliser soi-même.

Tokeniser : aucun découpage naïf ne suffit

Découper sur les espaces

```
"l'ordinateur"   → ["l'ordinateur"]
"aujourd'hui"    → ["aujourd'hui"]
"porte-monnaie"  → ["porte-monnaie"]
```

1 token chacune :

l'apostrophe et le trait d'union ne sont pas vus comme des frontières.

Découper avec `\w+`

```
re.findall(r"\w+", "cœur été l'ordinateur")
→ ['cœur', 'été', 'l', 'ordinateur']
```

Les accents et `œ` passent.

Mais `l'ordinateur` est coupé en deux, et `\w` inclut chiffres et `_` .

Aucune règle naïve n'est correcte pour toutes les tâches : le bon découpage dépend de l'**unité** qu'on veut manipuler.

Les stop words : un gain, un piège

Le gain

- moins de dimensions donc des calculs plus rapides
- moins de bruit : les mots outils n'occupent plus l'essentiel des comptages

Le piège

`je ne veux pas payer`

→ après retrait : `veux payer`

La négation a disparu. Le sens est inversé.

- Souvent retirés pour **BoW / TF-IDF** — mais c'est un **hyperparamètre** : sur des avis clients, `ne…pas` porte le signal.
- Un tokenizer de Transformer découpe avant tout filtrage : on ne supprime rien avant lui.

Stemming ou lemmatisation ?

Mot	Stemming (stem produit)	Lemmatisation
mangeons	`mang`	`manger`
mangera	`mang`	`manger`
mangé	`mang`	`manger`
mangeais	`mang`	`manger`
meilleur	`meilleur`	`bon`

Stemming — règles de troncature (Snowball), sans dictionnaire.

Un stem n'est pas un radical linguistique : c'est la forme produite par l'algorithme.

Lemmatisation — analyse morphologique et étiquette grammaticale.

Plus lente ; `meilleur → bon` dépend de l'analyseur.

Étiqueter la grammaire : le POS tagging

Chaque token reçoit une étiquette grammaticale (Universal POS). Exemple :

token	étiquette
Le	`DET`
président	`NOUN`
parle	`VERB`
de	`ADP`
la	`DET`
banque	`NOUN`

À quoi ça sert

- **lever une ambiguïté** : `la fermeture` (nom) vs `fermer` (verbe)
- **lemmatiser juste** : `suis` → `être` ou `suivre` selon le contexte
- **préparer l'extraction d'entités** (séance 6)

C'est un **outil de pipeline**, pas un concept de ce cours : on l'utilise, on ne l'implémente pas.

Ce qu'on garde pour la suite

1 · Les sous-mots (BPE, WordPiece) — avec les sous-mots, « token » ne veut plus dire « mot » : c'est un morceau de mot, et ce sera le sens du terme en séance 6.

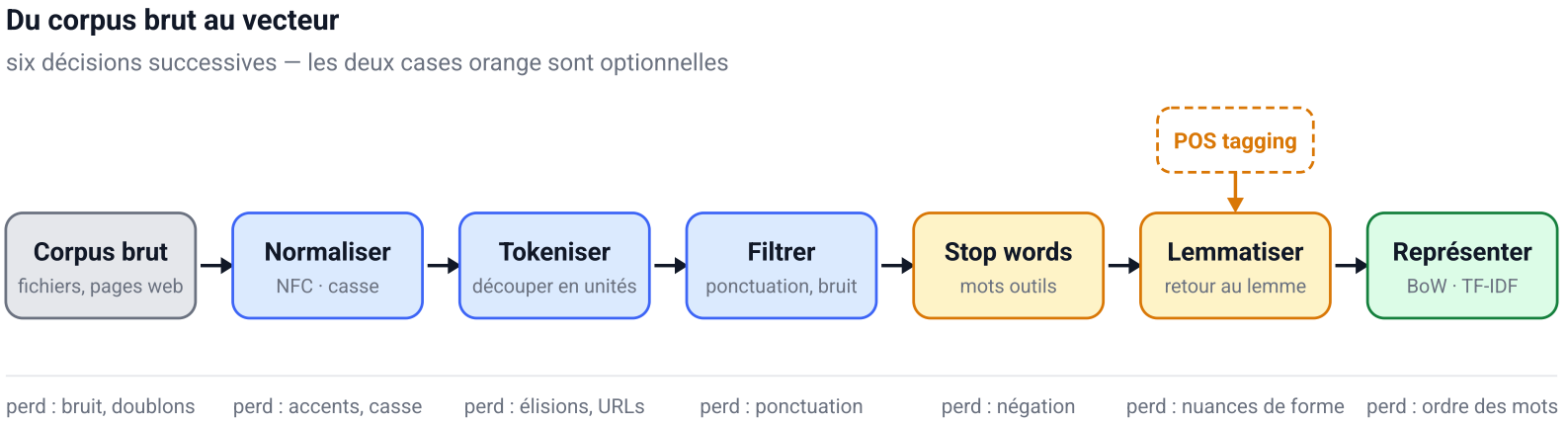
`incroyable` → `in` + `croy` + `able` (segmentation illustrative) : un mot inconnu devient une suite de morceaux connus.
C'est la réponse au problème des mots hors vocabulaire. Le principe est repris par tous les Transformers — BPE, WordPiece ou Unigram selon le modèle (séance 6).

2 · Le POS tagging, en trois lignes

Chaque token reçoit une étiquette (`DET` , `NOUN` , `VERB` … en Universal POS). Utile pour lever une ambiguïté et pour lemmatiser correctement. C'est un **outil de pipeline**, pas un concept de ce cours.

On sait maintenant produire des tokens propres.
Question suivante : comment dire que **deux textes se ressemblent** ?

Le pipeline de prétraitement, de bout en bout



Chaque étape est un choix : on ne « nettoie » pas, on **décide ce qu'on perd**.

Les deux étapes optionnelles (orange) ne le sont pas pour les mêmes raisons : les stop words allègent le vecteur, la lemmatisation rapproche les formes d'un même mot.

Le POS tagging n'est pas une étape de filtrage : il fournit l'étiquette dont la lemmatisation a besoin (`suis` → `être` ou `suivre`).

Ordre à respecter : on ne filtre pas avant d'avoir tokenisé, et on ne lemmatise pas sans étiquette grammaticale.

Un exemple, étape par étape

Phrase de départ : `Les chats d'Emily dorment sur le canapé du salon.`

Étape	Résultat
1. Normaliser	`les chats d'emily dorment sur le canape du salon`
2. Tokeniser	`["les", "chats", "emily", "dorment", "sur", "le", "canape", "du", "salon"]`
3. Filtrer	rien à retirer ici (pas de ponctuation, pas de chiffre)
4. Stop words	`["chats","emily","dorment","canape","salon"]`
5. Étiqueter (POS)	`chats/NOUN · dorment/VERB · canape/NOUN · salon/NOUN`
6. Lemmatiser	`["chat","emily","dormir","canape","salon"]`

C'est la liste finale qui entre dans le dataset — pas la phrase d'origine.

Ce qu'on garde dans le dataset

id	texte brut	tokens	n
1	Les chats d'Emily dorment sur le canapé.	`chat emily dormir canape`	4
2	Le chien aboie dans le jardin.	`chien aboyer jardin`	3
3	Un félin se repose sur le sofa.	`felin reposer sofa`	3

- **Toujours garder** la colonne `texte brut` : sans elle, impossible de revenir en arrière si le nettoyage a détruit une information
- **Une ligne = un document** (une phrase, un paragraphe, un avis…)
- **Colonnes en plus** selon la tâche : étiquettes, POS, longueur, date

Le nettoyage se choisit selon la tâche. Pour de la classification d'avis, on garde `ne… pas` . Pour de la recherche par mots-clés, on retire les stop words.

Deux chaînes se ressemblent-elles ?

Une faute de frappe — `ordinateur` / `ordinatuer`
L'utilisateur a tapé trop vite.

Une flexion — `carton` / `cartons`
Le même objet, deux formes.

Un synonyme — `voiture` / `automobile`
Le même objet, et pourtant une distance de **8**.

Peut-on mesurer la ressemblance *sans* passer par des vecteurs ? Commençons par le plus simple : les caractères.

Trois opérations, un coût de 1

Insertion

`chat` → `chats`

ajouter `s`

Suppression

`chats` → `chat`

ôter `s`

Substitution

`chat` → `chot`

remplacer `a` par `o`

Distance de Levenshtein = coût minimal de la suite d'opérations qui transforme une chaîne en l'autre.

La récurrence

$$\text{lev}(i,j) = \min \begin{cases} \text{lev}(i-1,j)+1 & \text{supprimer } a_i \\ \text{lev}(i,j-1)+1 & \text{ajouter } b_j \\ \text{lev}(i-1,j-1)+[a_i \neq b_j] & \text{substituer, ou conserver si } a_i = b_j \end{cases}$$

$$\text{conditions initiales :} \quad \text{lev}(i,0) = i \qquad \text{lev}(0,j) = j$$

- $\text{lev}(i,j)$ = distance entre les i **premiers caractères** de a et les j **premiers** de b.
- $[a_i \neq b_j]$ vaut **0 si les caractères sont égaux, 1 sinon**.
- On lit le résultat dans la **dernière case** de la matrice.
- Les conditions initiales ne sont pas décoratives : sans elles, la récurrence ne démarre pas.

La matrice : initialisation

a = `chat` (lignes) · *b* = `chats` (colonnes) · la première ligne et la première colonne sont les conditions initiales.

	ε	c	h	a	t	s
ε	0	1	2	3	4	5
c	·	·	·	·	·	·
h	·	·	·	·	·	·
a	·	·	·	·	·	·
t	·	·	·	·	·	·

Transformer 0 caractère en *j* caractères coûte *j* insertions — et inversement.

La matrice : on remplit case par case

Chaque case regarde ses **trois voisines** (haut, gauche, diagonale) et prend le minimum. Ici le calcul de la case ligne `a` , colonne `a` .

	ε	c	h	a	t	s
ε	0	1	2	3	4	5
c	1	0	1	2	3	4
h	2	1	0	1	2	3
a	3	2	1	**0**	1	2
t	·	·	·	·	·	·

haut = 1+1 = 2 · gauche = 1+1 = 2 · diagonale = 0+0 = **0** (mêmes caractères) → **0**

La matrice : le résultat

La dernière case donne la distance : **1**. Le chemin surligné montre l'opération retenue.

	ε	c	h	a	t	s
ε	**0**	1	2	3	4	5
c	1	**0**	1	2	3	4
h	2	1	**0**	1	2	3
a	3	2	1	**0**	1	2
t	4	3	2	1	**0**	**1**

`chat` → `chats` : on conserve `c`, `h`, `a`, `t`, puis on **insère** `s`. Coût total : **1**.

Normaliser la distance, et les autres cas

Similarité normalisée :

$$1 - \frac{\text{lev}(a, b)}{\max(|a|, |b|)}$$

Paire	Distance	Similarité
`chats` / `chat`	1	0,80
`carton` / `cartons`	1	0,86
`chien` / `chats`	3	0,40
`ordinateur` / `ordinatuer`	2	0,80
`voiture` / `automobile`	8	0,20

La formule de normalisation ci-dessus est une **convention** parmi d'autres (on divise aussi parfois par la somme des longueurs). En **Damerau-Levenshtein**, une transposition compte pour 1 : `ordinatuer` revient alors à une distance de 1.
Par convention, la similarité de deux chaînes vides vaut 1.

Où ça marche, où ça casse

Ça marche

- correction orthographique
- dédoublonnage de fichiers ou de contacts
- recherche approximative (fuzzy)
- alignement de séquences (avec des coûts adaptés)

Ça casse

- `voiture / automobile` : distance 8 sur 10, sens identique
- aucune notion de mot ni de sens
- coût $O(nm)$ sur de longs textes
- comparer des documents est possible, mais coûteux et rarement pertinent

On a mesuré des **caractères**. Pour comparer des **textes**, il faut changer d'unité : les mots.

L'idée du sac de mots

Un document devient un **sac** : on garde *quels* mots et *combien* de fois, on jette l'ordre.

> « le chat dort sur le canapé du salon » → `chat` ×1 · `dort` ×1 · `canapé` ×1 · `salon` ×1 · `le` ×2 · `sur` ×1 · `du` ×1

Le vocabulaire

Sur nos 4 documents : **18 termes** — autant de types distincts.
C'est l'ensemble des colonnes de la matrice.

La question

Deux sacs étant donnés, comment dire qu'ils se ressemblent — avec un **nombre**, pas une intuition ?

Le sac, mécaniquement

Phrase 1

le	chat	dort	sur	le	canapé

Phrase 2 — les mêmes mots, autre ordre

sur	le	canapé	dort	le	chat

on jette tout dans un sac



Ce qu'on garde : les comptes

le	2
chat	1
dort	1
sur	1
canapé	1

vecteur = [2, 1, 1, 1, 1, 0, 0, …]



Deux phrases qui disent la même chose dans un autre ordre donnent **le même vecteur** : on perd l'ordre, on gagne un comptage comparable.
Le vecteur a **une case par terme du vocabulaire** : ici 5 termes, sur les 18 du corpus — les autres cases valent 0. La dimension est fixe, même si la phrase est courte.

Le sac, au sens littéral : on y jette des morceaux, on perd leur disposition. Photo : Meritxell.canela, Wikimedia Commons, domaine public.

Remplir les cases : présence ou comptage ?

Une seule phrase : `le chat dort, le chat dort encore` — vocabulaire de 4 termes.

Présence (0 / 1)

le	chat	dort	encore
1	1	1	1

Le modèle sait que les mots sont là — pas combien de fois.

Comptage

le	chat	dort	encore
2	2	2	1

La répétition est conservée : c'est cette version que TF-IDF utilisera.

Même phrase, même vocabulaire, **deux vecteurs différents** : le choix du score fait partie du modèle, pas du texte.

D'après J. Brownlee, *A Gentle Introduction to the Bag-of-Words Model*, Machine Learning Mastery, 2019.

La matrice documents × termes

Lignes = documents · colonnes = termes du vocabulaire · valeur = nombre d'occurrences. 10 colonnes montrées sur 18.

document	chat	chien	dort	canapé	tapis	félin	sofa	salon	le	est	…
D1	1	0	1	1	0	0	0	1	2	0	…
D2	0	1	1	0	1	0	0	1	2	0	…
D3	0	0	0	0	0	1	1	0	0	0	…
D4	0	0	0	0	0	0	0	2	2	2	…

Chaque ligne est un **vecteur de 18 nombres**. La plupart valent zéro.

Un vecteur très creux

Ici

- 18 dimensions
- **6 ou 7** valeurs non nulles par document
- densité moyenne **36 %** (26 cases sur 72) — parce que le corpus est minuscule

En réel

- 10^4 à 10^6 dimensions
- quelques dizaines de mots par document
- densité **< 1 %**

> Sur un vrai corpus, un vecteur de 50 000 dimensions contient souvent moins de 100 valeurs non nulles. C'est ce qu'on appelle un vecteur **creux** (*sparse*).

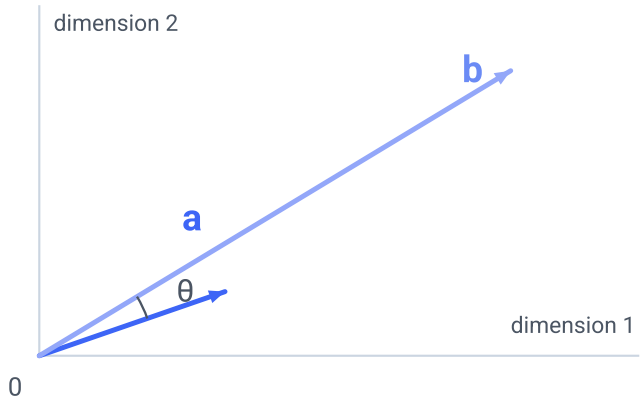
`fit` puis `transform` : la règle d'or

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)   # apprend le vocabulaire ET transforme
x = vectorizer.transform([requete])    # projette DANS ce vocabulaire
```

- `fit` **fige** la liste des colonnes — jusqu'au prochain `fit`, qui crée un autre modèle.
- `transform` ignore silencieusement les mots absents du vocabulaire.
- Conséquence : une requête peut très bien ne produire **aucun** signal.

> Le vocabulaire fait partie du **modèle**. Recharger un modèle sans son vocabulaire, c'est comparer deux espaces différents.

Similarité cosinus



$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

- $\cos = 1$: même direction
- $\cos = 0$: vecteurs orthogonaux — aucun mot partagé
- ici jamais négatif : les comptages sont **non négatifs**

Pourquoi pas la distance euclidienne ? Elle est sensible à la **norme** : q et $2q$ désignent la même direction, mais sont à distance non nulle. Le cosinus ne regarde que l'**angle**.

La norme utilisée est la norme L2 : $\|\mathbf a\|=\sqrt{\sum_i a_i^2}$.
Si l'un des deux vecteurs est nul, le cosinus est **indéfini** (0/0).

Exemple : requête « le chat »

Document	Contenu	Cosinus
D1	le chat dort sur le canapé du salon	**0,671**
D2	le chien dort sur le tapis du salon	0,447
D3	un félin se repose sur un sofa	0,000
D4	le salon est grand et le salon est clair	0,365

- D1 gagne : $q \cdot D1 = 1 \times 2 + 1 \times 1 = 3$, $\|q\| = \sqrt{2}$, $\|D1\| = \sqrt{10} \rightarrow 3/\sqrt{20} = 0{,}671$.
- D2 et D4 ont le **même produit scalaire** ($q \cdot D = 2$) : c'est la norme qui tranche, $\sqrt{10}$ contre $\sqrt{15}$ → 0,447 contre 0,365.
- D3 tombe à **0** : aucun mot commun. Premier signal d'alarme.

Le sac perd l'ordre — et le sens

`le chien mord l'homme`

`l'homme mord le chien`

Mêmes comptages, **cosinus = 1**.

Deux scènes opposées, un score parfait.

La parade classique : les bigrammes

`le chien` , `chien mord` , `mord l'homme` … On réintroduit un **ordre local entre mots adjacents**, au prix d'un espace de bigrammes théoriquement $V \times V$.

> Les bigrammes ajoutent de l'ordre local. Ils ne règlent **ni** les synonymes (`voiture` / `automobile` : cosinus 0) **ni** les mots inconnus.

Et si aucun mot n'est connu ?

Requête

```
animal de compagnie
```

Vocabulaire du corpus

```
chat , chien , dort , canapé , tapis ,
félin , sofa , salon …
```

> Aucun terme commun → **vecteur nul** → cosinus **indéfini** (0/0), que scikit-learn affiche à 0 par convention.

Ce n'est pas un bug : un mot absent du vocabulaire n'a aucune colonne où s'écrire.
C'est pourtant le cas le plus fréquent en production : vocabulaire métier, fautes, abréviations, autre langue.

Le comptage brut a un défaut

Mot	Occurrences	Documents concernés
`le`	6	3 sur 4
`salon`	4	3 sur 4
`félin`	1	1 sur 4

`le` est présent dans 3 documents sur 4 : il ne peut, au mieux, que distinguer D3 des trois autres. `félin` n'apparaît que dans **un seul** document : il isole exactement la bonne réponse.

Il faut donc **donner un poids** aux termes — et la bonne mesure de « rare » n'est pas la fréquence dans le document, mais le **nombre de documents qui contiennent le mot**. C'est tout l'objet de TF-IDF.

Le sac de mots : résumé

- **Vecteur** : une case par mot du vocabulaire
- **Valeur** : présence (0/1) ou nombre d'occurrences
- **Cosinus** : compare la direction de deux vecteurs
- **Limites** : ordre perdu, synonymes séparés, mots inconnus ignorés

Une requête sans aucun mot connu donne un vecteur nul — et un cosinus qui ne veut rien dire.

L'idée : donner plus de poids à ce qui distingue

Le problème

`le` est partout, donc il ne sépare rien.
`félin` est rare, donc il identifie.

Or le comptage brut les traite de la même façon : **1 occurrence = 1**.

L'idée

Un terme est important s'il est :

- **fréquent dans le document** → TF
- **rare dans le corpus** → IDF

> Intuition : un mot qui apparaît dans *tous* les documents ne peut pas servir à en choisir un.



Salton & Buckley (1988), *Term-weighting approaches in automatic text retrieval.*

Les deux facteurs

TF — nombre d'occurrences du terme dans le document (comptage brut dans scikit-learn) :

$$\mathrm{tf}(t, d) = n_{t,d}$$

IDF — à quel point le terme est rare dans le corpus (convention scikit-learn, lissée) :

$$\mathrm{idf}(t) = \ln\left(\frac{1 + N}{1 + \mathrm{df}(t)}\right) + 1 \qquad w_{t,d} = \mathrm{tf}(t, d) \times \mathrm{idf}(t)$$

N = nombre de documents · *df(t)* = nombre de documents contenant *t* · les *+1* lissent le rapport : l'IDF ne peut jamais être nul ni négatif.
Le vecteur obtenu est ensuite **normalisé en L2**. La variante « classique » divise *tf* par la longueur du document — un facteur **par document**. Après normalisation L2, les deux conventions donnent des cosinus **identiques**.

Calcul sur notre corpus

N = 4 documents. Valeurs calculées avec la formule lissée ci-dessus.

Terme	df	IDF
`le`	3	ln(5/4) + 1 = **1,223**
`salon`	3	ln(5/4) + 1 = **1,223**
`dort`	2	ln(5/3) + 1 = **1,511**
`chat`	1	ln(5/2) + 1 = **1,916**
`félin`	1	ln(5/2) + 1 = **1,916**

Aucun terme n'est jamais annulé : `le`, présent dans 3 documents sur 4, garde un IDF de 1,22, soit **1,57 fois moins** qu'un mot unique — mais jamais zéro.

Requête « le chat », cette fois en TF-IDF

Document	Cosinus (comptages)	Cosinus (TF-IDF)
D1	**0,671**	**0,641**
D2	0,447	0,288
D3	0,000	0,000
D4	0,365	0,214

- L'**ordre ne change pas** : D1 reste premier, et c'est normal — D1 contient vraiment `chat`.
- Ce qui change, c'est l'**écart** : D2 passe de 0,447 à 0,288, D4 de 0,365 à 0,214.
- TF-IDF ne réécrit pas la vérité : il **accentue les différences**.

Le cas où le classement bascule

Requête : `félin salon`

Document	Cosinus (comptages)	Cosinus (TF-IDF)
D1 le chat dort sur le canapé du salon	0,224	0,144
D2 le chien dort sur le tapis du salon	0,224	0,144
D3 un félin se repose sur un sofa	0,236	**0,291**
D4 le salon est grand et le salon est clair	**0,365**	0,214

- En **comptages**, D4 gagne : il répète `salon` deux fois, alors qu'il ne parle pas d'animaux.
- En TF-IDF, **D3 gagne** : `salon` est banalisé (3 documents sur 4), `félin` retrouve son poids.
- C'est exactement le comportement attendu : la pondération corrige un classement trompeur.

Les limites de TF-IDF

- **Synonymes** : `voiture` et `automobile` restent à distance
- **Plusieurs sens** : les deux sens d' `avocat` partagent une seule case
- **Ordre** : il n'est pas conservé
- **Mots inconnus** : ils sont ignorés
- **Corpus** : changer les documents change tous les poids

TF-IDF donne un poids aux mots, il ne comprend pas leur sens.

Récapitulatif

1. **TF** mesure la présence dans le document, **IDF** la rareté dans le corpus.
2. La convention scikit-learn est **lissée** : un mot présent partout garde un IDF de 1 — un **plancher**, pas un poids fort.
3. TF-IDF **accentue les écarts** ; il ne change l'ordre que quand un mot banal écrase un mot qui distingue.
4. Il reste **basé sur les mots** : synonymes, plusieurs sens et mots inconnus lui échappent.

Question — Pourquoi un IDF calculé sur 4 documents ne doit-il pas être réutilisé tel quel sur un corpus de 10 000 documents ?

Un one-hot ne dit rien

Le one-hot

Un vecteur de la taille du vocabulaire, avec un seul 1.

```
chat  = [1,0,0,0,0,...]
chien = [0,1,0,0,0,...]
pneu  = [0,0,0,1,0,...]
```

cos(chat, chien) = **0**

cos(chat, pneu) = **0**

Aucune information : tous les mots sont équidistants.

L'idée de base (Firth, 1957)

> « You shall know a word by the company it keeps. »

Deux mots qui apparaissent **dans les mêmes contextes** ont un sens proche.

C'est tout le programme de Word2Vec : apprendre des vecteurs **à partir des voisinages**, sans dictionnaire.

Mikolov et al. (2013), *Efficient Estimation of Word Representations in Vector Space*.

La fenêtre glissante

`le(0) chat(1) dort(2) sur(3) canapé(4)` — fenêtre $k = 2$

Cible : `dort` (position 2)

Contexte = mots à une distance ≤ 2 :

`le` (0) · `chat` (1) · `sur` (3) · `canapé` (4)

→ 4 associations mot ↔ contexte :

(`dort` , `le`), (`dort` , `chat`), (`dort` , `sur`),
(`dort` , `canapé`)

Puis on décale

Cible `le` (0) : contexte `chat` , `dort` (bornes !)

Cible `chat` (1) : contexte `le` , `dort` , `sur`

… et ainsi de suite sur tout le corpus.

La fenêtre est un **hyperparamètre** : petite = relations syntaxiques, grande = thèmes.

Deux façons d'apprendre : CBOW et Skip-gram

CBOW — prédire le centre depuis le contexte

```
contexte : le, chat, sur, canapé
         ↓
cible    : dort
```

Rapide, efficace sur les mots fréquents.

Skip-gram — prédire le contexte depuis le centre

```
cible    : dort
         ↓
contexte : le, chat, sur, canapé
```

Plus lent, meilleur sur les **mots rares**.

Les deux produisent la même chose : une matrice de vecteurs. Seul le sens des paires change.

Où sont les vecteurs ?

Chaque mot possède une ligne dans la **matrice d'entrée W** (taille $|V| \times d$) : c'est elle qui contient les embeddings.

$$h = \frac{1}{|C|} \sum_{w \in C} \mathbf{e}_w \qquad \text{avec } C = \text{contexte}(w_t),\ w_t \notin C,\ |C| \leq 2k$$

e_w = le vecteur du mot w (une ligne de W) · w_t = le mot central, la cible · h = le vecteur agrégé du contexte · |C| ≤ 2k parce qu'au premier mot de la phrase, la fenêtre ne contient que k voisins.

- **CBOW** : on moyenne les vecteurs du contexte pour former *h*, puis on calcule un score pour chaque mot du vocabulaire.
- **Skip-gram** : on part du vecteur du mot central et on calcule un score pour chaque mot du contexte.
- L'entraînement **ajuste W** : les mots réellement observés doivent obtenir un score élevé.

Combien ça coûte ?

Deux matrices

- entrée : $|V| \times d$
- sortie : $|V| \times d$

10 000 mots × 100 dimensions → **2 × 10⁶ poids**

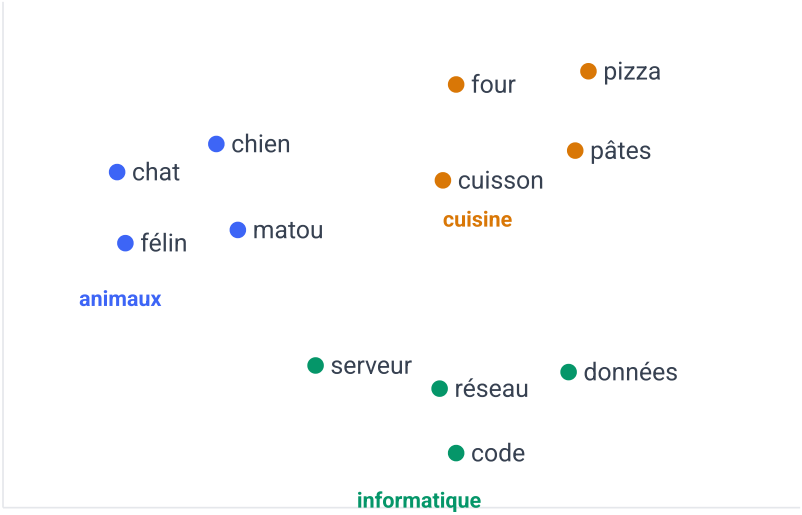
La **ligne de la matrice d'entrée** associée à un mot *est* son embedding : la couche intermédiaire n'est que la moyenne de ces lignes.

Ordres de grandeur

- corpus : millions de mots
- dimension : 100 à 300 — compromis capacité / coût ; au-delà, les gains deviennent marginaux
- vocabulaire : 10^4 à 10^6

Un vocabulaire de 100 000 mots en 300 dimensions = **60 millions** de paramètres, pour deux matrices. Et pour chaque paire, il faut calculer $|V|$ scores : c'est ce **coût de calcul** que réduisent le negative sampling et le softmax hiérarchique (séance 6), pas le nombre de paramètres.

Ce que ça donne

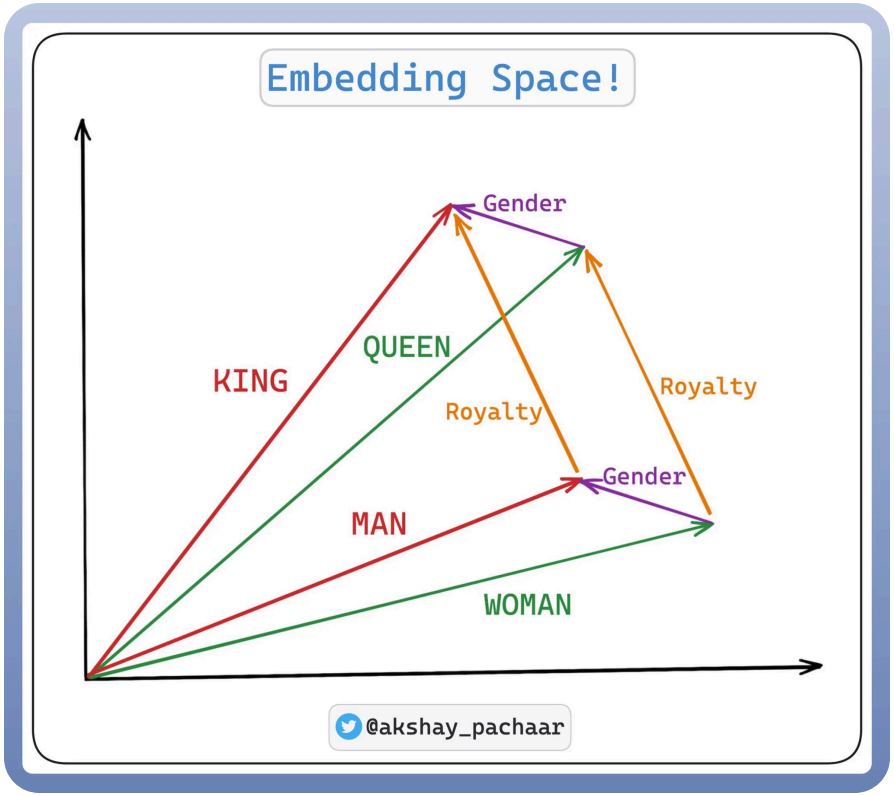


Après entraînement sur un gros corpus, les voisins de `chat` sont `chien`, `matou`, `félin` — avec des cosinus de 0,6 à 0,9.
Même famille sur le modèle du TP : `félin` 0,72 · `chien` 0,62 · `chats` 0,62.

Les thèmes se regroupent **sans qu'on les ait jamais nommés**.

Attention : c'est une similarité **de contextes**, pas du « sens ». `chaud` et `froid` sont proches (mêmes contextes) ; `canapé` et `sofa` aussi.

Lire un espace d'embeddings



1. Chaque mot est un **vecteur tracé depuis l'origine** (MAN = `homme`, WOMAN = `femme`, KING = `roi`, QUEEN = `reine`).

2. Les flèches violettes « Gender » sont **identiques** : MAN→WOMAN et KING→QUEEN.

3. Les flèches orange « Royalty » sont **identiques** : MAN→KING et WOMAN→QUEEN.

4. Les quatre points forment un **parallélogramme** :

```
QUEEN = KING + (WOMAN - MAN)
```

Un vecteur différence n'est pas un mot : c'est un **sens** — et il se réutilise.

Figure : schéma de @akshay_pachaar, repris tel quel avec son crédit d'auteur.

Ce que ça vaut vraiment

- **Ce qui marche** : `reine` sort en tête pour `roi - homme + femme`
- **Ce qui échoue** : `paris - france + italie` peut donner `espagne`
- **Le chiffre** : cos(`reine-roi` , `femme-homme`) = **0,37** sur notre modèle
- **Pourquoi** : un petit corpus donne des vecteurs moins fiables

Une analogie réussie ne prouve pas que le modèle a compris.

D'un mot à un document

Un mot a maintenant un vecteur. Mais une **phrase**, un **paragraphe**, un **document** en ont-ils un ?

Pooling
Moyenner les vecteurs des mots du document. Simple — mais l'ordre et les relations entre mots disparaissent.

Encodeur de phrases
Un modèle entraîné pour produire *directement* un vecteur de phrase.
C'est ce que fait `sentence-transformers` (séance 6).

C'est **ce** vecteur-là qui servira à comparer des phrases entières — pas les vecteurs de mots isolés. Tant que ce maillon n'est pas vu, la comparaison de documents reste une boîte noire.

Cinq limites, un même mur

Limite	Symptôme	Ce qu'il faudrait
Lexical (BoW, TF-IDF)	`voiture` / `automobile` → cosinus 0	des vecteurs appris
Polysémie	`avocat` = fruit **et** métier, un seul vecteur	un vecteur **par contexte**
Ordre	`le chien mord l'homme` ≈ `l'homme mord le chien`	modéliser la séquence
Hors vocabulaire	un mot absent n'a *aucun* vecteur	des sous-mots (fastText), un vrai encodeur
Expression	`animal de compagnie` : le sens n'est pas la somme des mots	un encodeur de phrases

Word2Vec **atténue** la première ligne, sans la supprimer. Les suivantes demandent de regarder **la phrase entière**, pas le mot isolé.

Un vecteur par contexte

- **Fruit** : « L'avocat mûr tombe de l'arbre. »
- **Métier** : « L'avocat plaide au tribunal. »
- **Word2Vec** : un seul vecteur pour `avocat`, les deux sens mélangés
- **BERT** : un vecteur différent dans chaque phrase

Le contexte sert à choisir le bon sens — c'est l'objet de la séance 6.

Les trois façons de représenter un texte

	Ce qu'elle garde	Ce qu'elle perd	Coût
Comptages (BoW)	quels mots, combien de fois	l'ordre, les synonymes	très faible
TF-IDF	les mots qui distinguent	l'ordre, les synonymes	faible
Word2Vec	le sens approximatif	l'ordre, la précision	moyen

À retenir : on ne remplace pas une méthode par une autre — on choisit celle qui correspond à la question posée.

Une recherche par mots-clés exacts ? BoW ou TF-IDF suffit. Une recherche où l'utilisateur écrit autrement que le document ? Il faut Word2Vec.

Exercice : trois phrases, trois vecteurs

Corpus de trois phrases :

- `D1` le chat dort sur le canapé
- `D2` le chien dort sur le tapis
- `D3` un félin se repose sur le sofa

1. Vectoriser en comptages.

2. Calculer le cosinus entre D1 et D2, puis entre D1 et D3.

3. Refaire le calcul en TF-IDF.

4. Expliquer pourquoi D3 change de place.

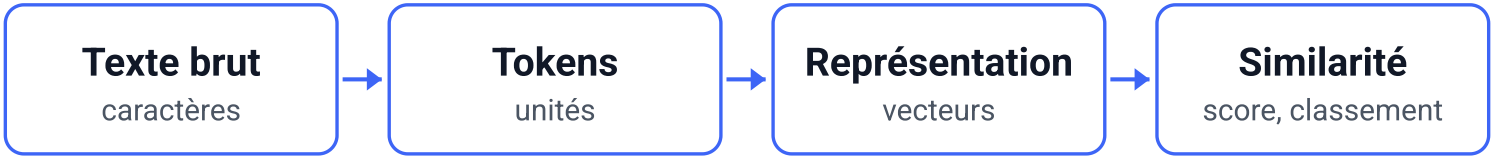
Attendu : D1 et D2 se ressemblent (mêmes mots outils, `dort` en commun). D3 parle de la même chose sans partager un seul mot de contenu — les comptages ne peuvent pas le voir.

Ce qu'il faut savoir refaire à la main

- **1.** Compter les occurrences et construire la matrice documents × termes.
- **2.** Calculer un IDF : `log(N / df)` — et dire pourquoi il monte quand le mot est rare.
- **3.** Calculer un cosinus entre deux vecteurs, y compris le cas du vecteur nul.
- **4.** Remplir une matrice de Levenshtein sur 4 × 4 cases.
- **5.** Trouver les voisins d'un mot dans un fichier de vecteurs.

Si ces cinq gestes sont acquis, la suite (phrase, document, recherche) n'est plus qu'une combinaison de ce que vous savez déjà.

La carte du cours



Le contexte, lui, sera encodé dans la représentation à partir de la séance 6.

- **Levenshtein** : compare les caractères, sans dictionnaire
- **BoW / TF-IDF** : comptent les mots et donnent un poids aux mots rares
- **Word2Vec** : rapproche les mots employés dans les mêmes contextes
- **Toutes** perdent quelque chose : ordre, synonymes ou mots inconnus

Il n'existe pas de « meilleure représentation » : il existe une représentation adaptée à une question, avec un prix connu.

Trois situations, trois choix

Pour chacune, quelle représentation choisissez-vous — et quelle limite acceptez-vous ?

1. Chercher `voiture` dans un catalogue qui n'écrit que `automobile`.

2. Retrouver les documents où `avocat` désigne le fruit, pas le métier.

3. Répondre à une question dont *aucun* mot n'appartient au vocabulaire du corpus.

Attendu : une représentation nommée (BoW, TF-IDF, Word2Vec, encodeur de phrase), une justification, et la limite assumée.

Le TP : 3 blocs, 1 heure

Bloc 1 · 15 min
Normaliser, tokeniser, filtrer les stop words. Mesurer l'effet de chaque choix.

Bloc 2 · 20 min
BoW → TF-IDF → cosinus. Classer une requête, provoquer un vecteur nul.

Bloc 3 · 12 min
Embeddings pré-calculés : voisins, analogie, un échec à expliquer.

Total des blocs : 47 min — les **13 minutes restantes** sont consacrées à la lecture des consignes, aux erreurs d'exécution et aux questions.

À savoir faire en sortant

- expliquer pourquoi `voiture` et `automobile` ont un cosinus de 0 en TF-IDF
- dire ce qu'un `fit` fige, et ce qu'un `transform` ignore
- choisir entre comptage et vecteurs denses pour une tâche donnée

→ Place au TP.